VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

Bhopal-Indore Highway, Kothrikalan, Sehore

Madhya Pradesh - 466114

CSA1021 INTRODUCTION TO PROBLEM SOLVING

AND PROGRAMMING

REG.NO : 26BCE10233

NAME : Tamanna Arora

BRANCH : CSE Core

SEMESTER: Fall Semester 2026-27

INTRODUCTION

Student Study & Task Manager is a simple Python-based command-line application designed to help students organize and track their study tasks. The application provides a single place where users can add study tasks, view tasks, search for a task, mark tasks as completed, and check overall study progress.

The project demonstrates fundamental programming concepts including classes and objects, functions, lists, loops, conditional statements, user input, modular programming, input validation, and basic automated testing.

Project objectives

- *Provide a simple way to organize study-related tasks.*
- *Allow students to track pending and completed tasks.*
- *Provide search functionality for finding a task.*
- *Display overall task completion progress.*
- *Demonstrate modular Python programming and basic software testing.*

PROBLEM STATEMENT

College students often have multiple study tasks, assignments, and revision activities to complete. Managing these tasks manually can make it difficult to remember pending work and track study progress.

The Student Study & Task Manager is a Python-based application that helps students organize their study-related tasks in one place. Users can add tasks, view their tasks, search for a particular task, mark tasks as completed, and check their overall study progress.

Scope

The project provides a simple command-line system for managing study tasks and tracking completion progress. It focuses on basic task organization using Python.

Target Users

- *College students*
- *Students preparing for examinations*
- *Students who want to organize their study tasks*

FUNCTIONAL REQUIREMENTS

The system shall provide the following functional requirements :

FR NO.	Functional Requirements
FR1	Add a new study task.
FR2	View all tasks and their completed / pending status
FR3	Search for a task using keyboard.
FR4	Mark a task as completed.
FR5	Calculate and display study progress as a percentage.
FR6	Allow the user to exit the application.

NON FUNCTIONAL REQUIREMENTS

- **Usability:** *The system should have a simple and easy-to-understand command-line interface so that students can use it easily.*

- **Reliability:** *The system should correctly add, search, complete, and display study tasks without unexpected errors.*

- **Maintainability:** *The program should be divided into separate modules so that the code can be easily understood, maintained, and modified***.**

● **Error Handling:** *The system should handle invalid inputs such as empty task names and display suitable messages to the user.*

● **Performance:** *The system should perform task operations quickly for a normal number of study tasks.*

SYSTEM ARCHITECTURE

The application follows a simple modular architecture. The user interacts with main.py, which controls the program flow and communicates with the supporting modules.

Architecture Flow

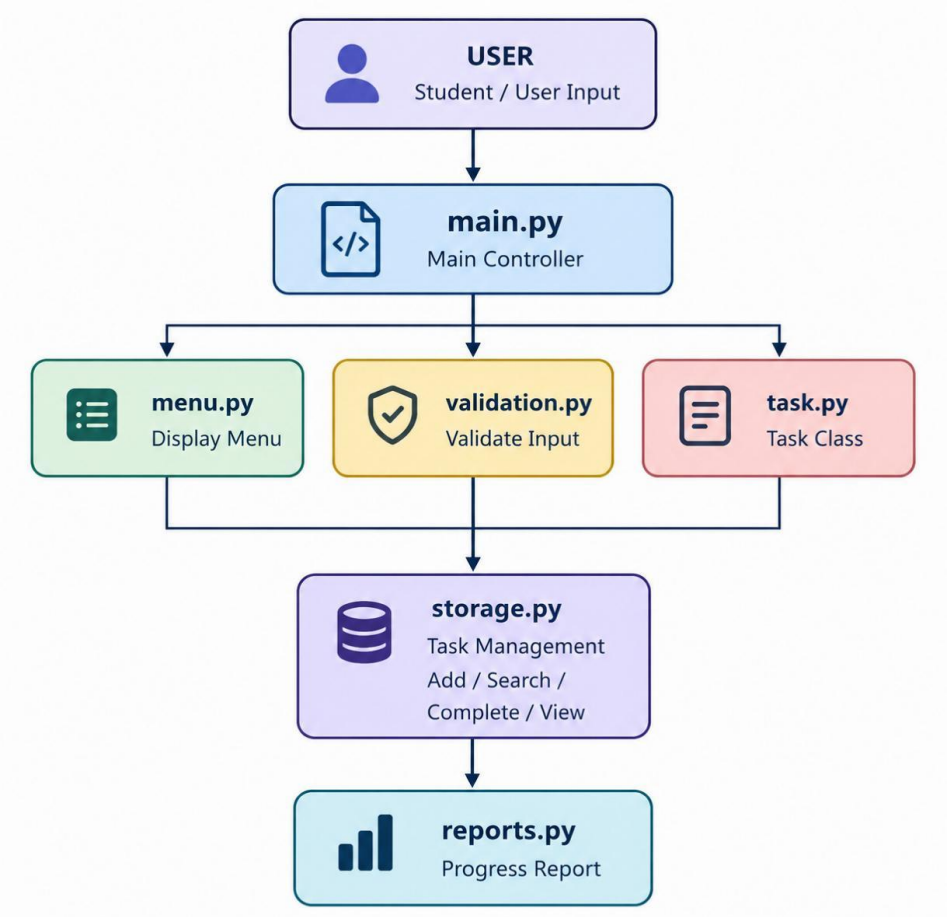


Figure 1: System Architecture of Student Study & Task Manager

MODULE RESPONSIBILITIES

MODULE	RESPONSIBILITY
main.py	Controls the main menu and overall program flow
task.py	Defines the Task class and task attributes.
storage.py	Manages the in-memory task list, adding, searching, viewing, and completing tasks.
validation.py	Validates task titles and handles empty/overly long task names.
reports.py	Calculates and displays total, completed, and percentage progress.
menu.py	Displays the command-line menu.
tests / test_task_manager.py	Contains automated tests using Python unit test.

DESIGN DIAGRAMS

Workflow Diagram

The workflow begins when the student starts the program. The main menu is displayed and the student selects an operation. The selected operation is performed, the result is displayed, and the program returns to the main menu until the user chooses Exit.

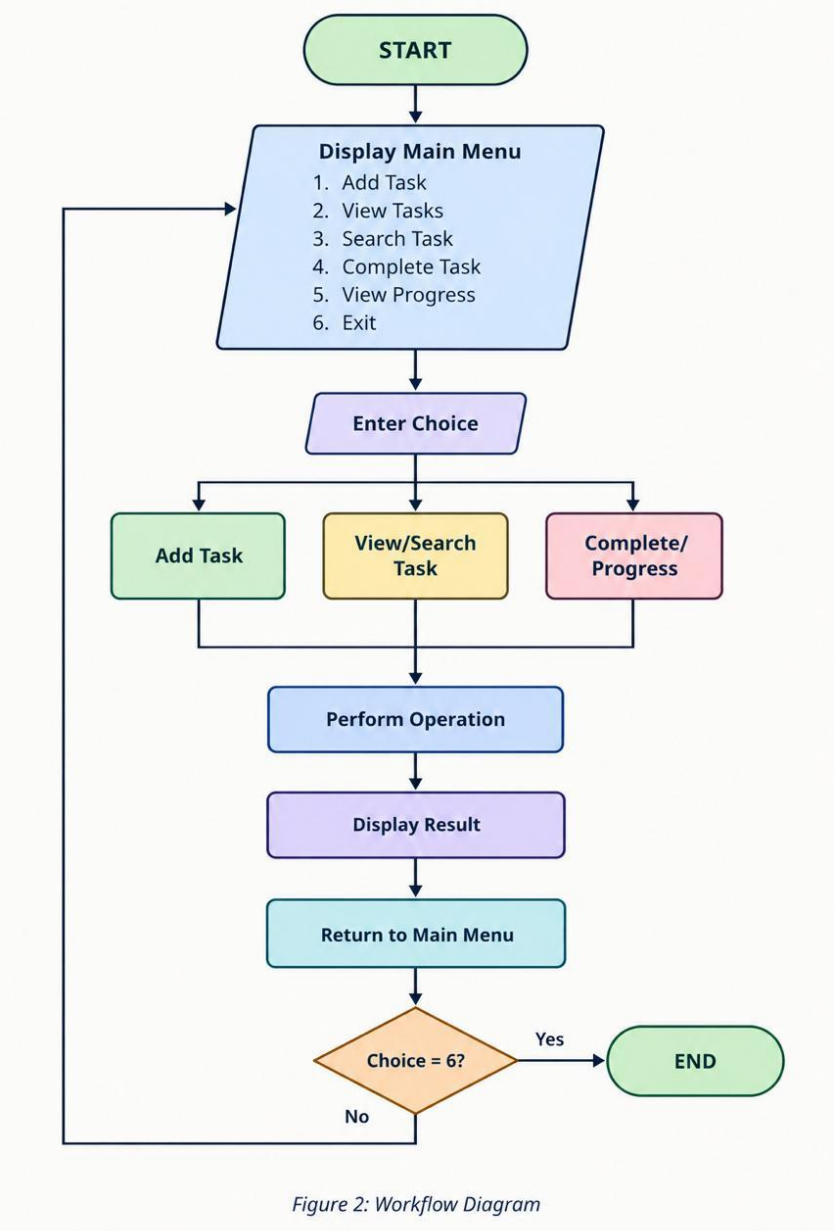


Figure 2: Workflow Diagram

Use case diagram

Actor: Student / User

• *Add Task*
• *View Tasks*
• *Search Task*
• *Mark Task Completed*
• *View Study Progress*

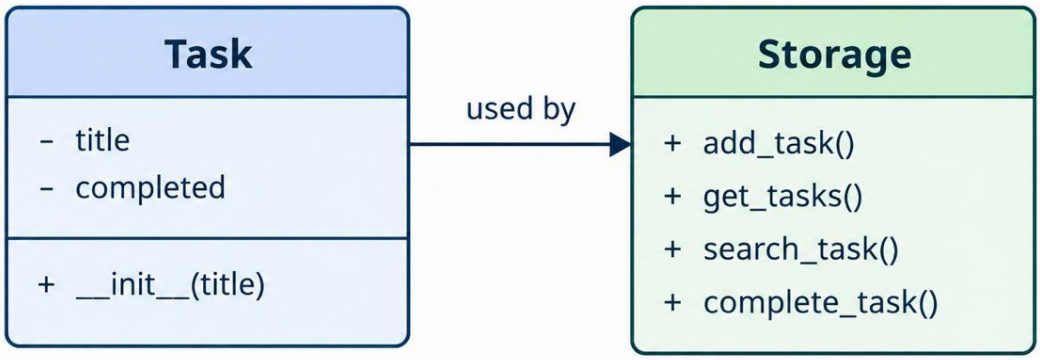


Figure 3: Class Diagram

Sequence Diagram

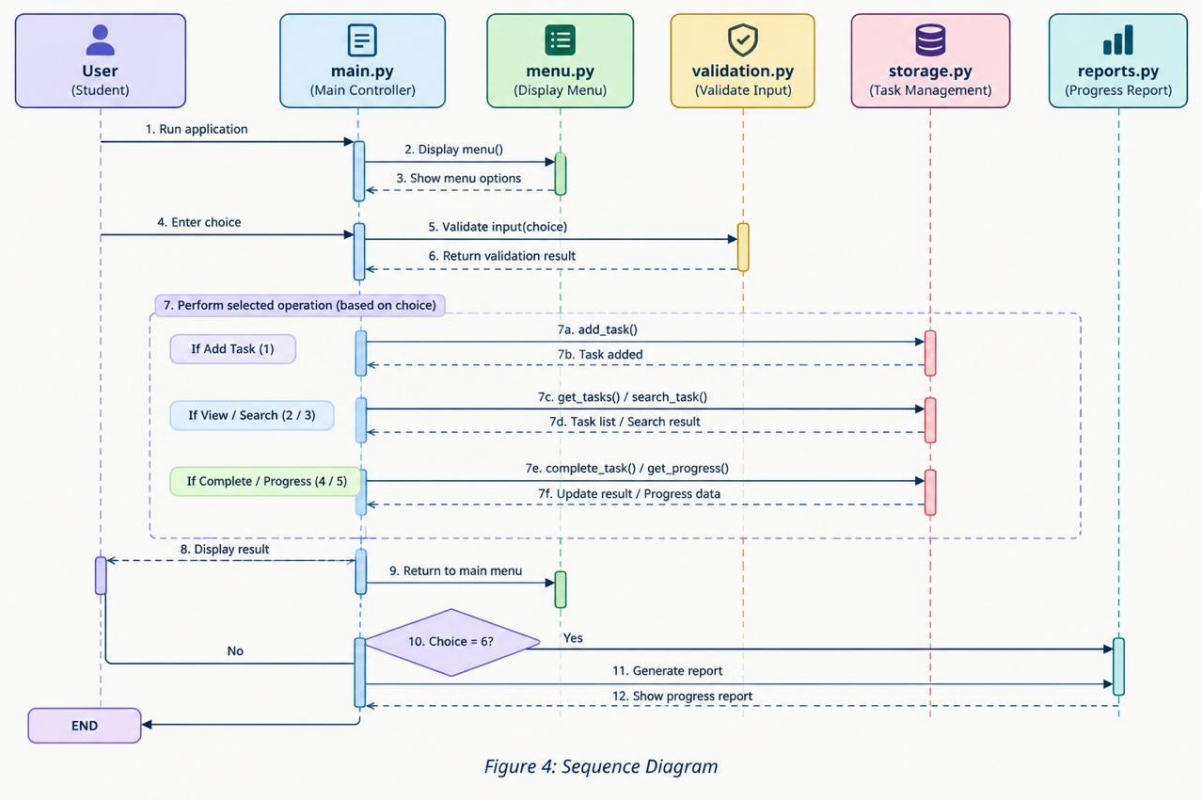


Figure 4: Sequence Diagram

STORAGE DESIGN

The current application uses an in-memory Python list named tasks. A separate database is not used. Tasks remain available while the program is running and are not permanently stored after the application closes.

DESIGN DECISIONS AND RATIONALE

• **Command-line interface:** *A CLI keeps the project simple and focuses on core Python programming concepts.*

• **Modular structure:** *Separate files divide responsibilities and improve maintainability*.

• **Task class:** *A class is used to represent each study task with its title and completion status.*

• **In-memory list:** *A Python list is sufficient for the current project scope and avoids unnecessary external database dependencies.*

• **Validation module:** *Input validation prevents empty task titles and limits excessively long titles.*

• **Automated tests:** *Python's built-in unit test framework provides a simple way to verify important program behavior.*

IMPLEMENTATION DETAILS

Technologies Used

• *Python 3*

• *Python classes and objects*

• *Lists*

• *Functions*

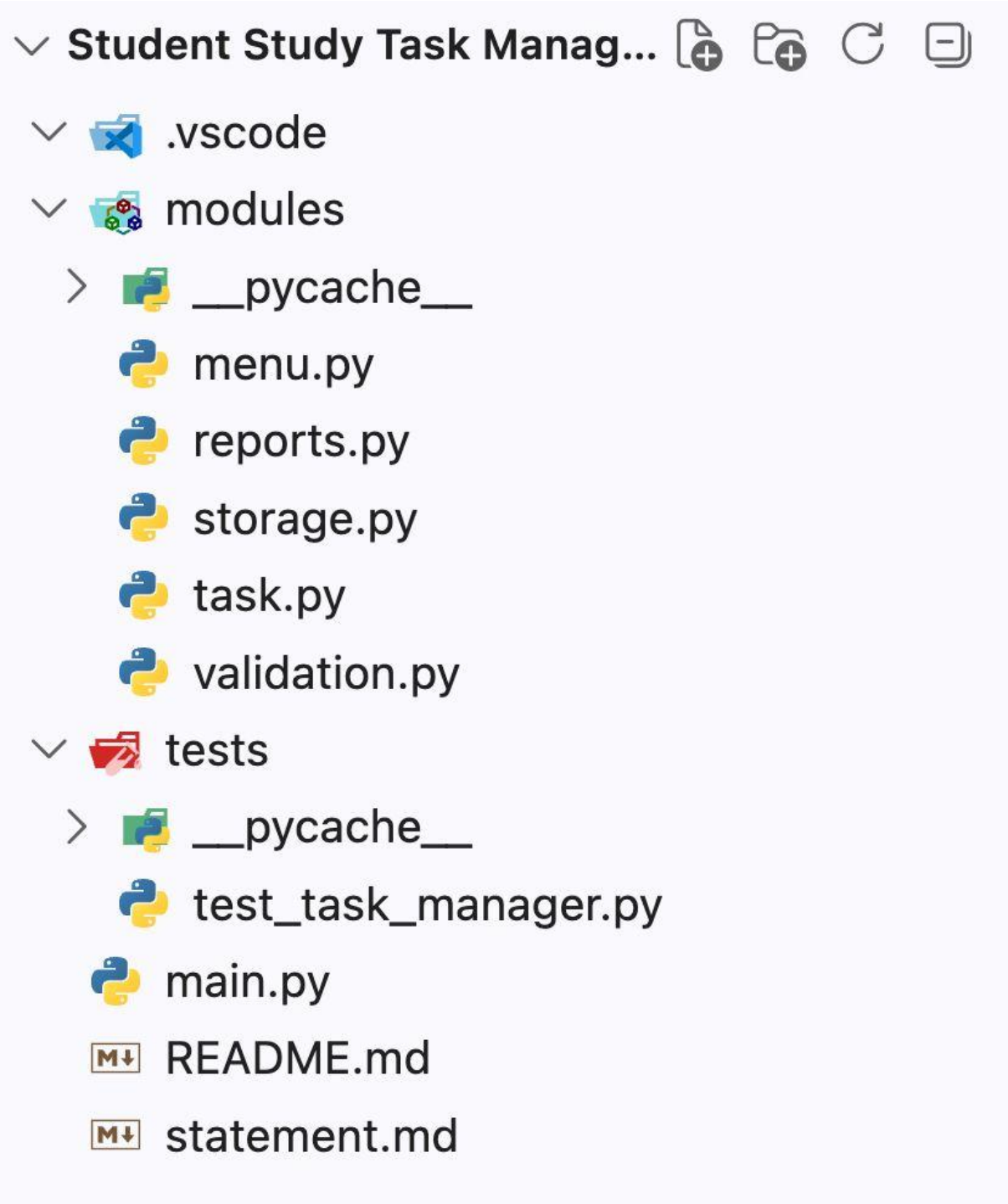
• *Loops*

• *Conditional statements*

• *Command-line interface*

• *unit test for automated testing*

Project Structure



Progress Calculation

Progress = (Completed Tasks / Total Tasks) × 100

SCREENSHOTS OF CODE

The following outputs were verified during manual testing of the application.

SCREENSHOT 1 – task.py

```python
class Task:
    def __init__(self, title):
        self.title = title
        self.completed = False
```

SCREENSHOT 2 – storage.py

```python
tasks = []


def add_task(task):
    tasks.append(task)


def get_tasks():
    return tasks


def search_task(keyword):
    found = []

    for task in tasks:
        if keyword.lower() in task.title.lower():
            found.append(task)

    return found


def complete_task(title):
    for task in tasks:
        if task.title.lower() == title.lower():
            task.completed = True
            return True

    return False
```

SCREENSHOT 3 – validation.py

```python
def validate_task_title(title):
    title = title.strip()

    if not title:
        return False, "Task title cannot be empty."

    if len(title) > 100:
        return False, "Task title must be 100 characters or fewer."

    return True, title
```

SCREENSHOT 4 – reports.py

```python
def show_progress(tasks):
    total = len(tasks)
    completed = 0

    for task in tasks:
        if task.completed:
            completed += 1

    print("Total tasks:", total)
    print("Completed tasks:", completed)

    if total > 0:
        percentage = (completed / total) * 100
        print("Progress:", round(percentage, 2), "%")
    else:
        print("Progress: 0 %")
```

SCREENSHOT 5 – menu.py

```python
def display_menu():
    print("\n===== Student Study Task Manager =====")
    print("1. Add Task")
    print("2. View Tasks")
    print("3. Search Task")
    print("4. Mark Task Completed")
    print("5. View Progress")
    print("6. Exit")
```

SCREENSHOT 6 – main.py

```python
from modules.task import Task
from modules.storage import add_task, get_tasks, search_task, complete_task
from modules.validation import validate_task_title
from modules.reports import show_progress
from modules.menu import display_menu


while True:
    display_menu()

    choice = input("Enter your choice: ").strip()

    if choice == "1":
        title = input("Enter your study task: ")

        valid, result = validate_task_title(title)

        if valid:
            task = Task(result)
            add_task(task)
            print("Task added successfully!")
        else:
            print(result)

    elif choice == "2":
        print("\nYour study tasks:")

        tasks = get_tasks()

        if not tasks:
            print("No tasks added yet.")
        else:
            for task in tasks:
                if task.completed:
                    print("-", task.title, "[Completed]")
                else:
                    print("-", task.title, "[Pending]")

    elif choice == "3":
        keyword = input("Enter a keyword to search: ")

        print("Search results:")

        results = search_task(keyword)

        if results:
            for task in results:
                print("-", task.title)
        else:
            print("No matching task found.")

    elif choice == "4":
        title_to_complete = input("Enter the task you completed: ")

        if complete_task(title_to_complete):
            print("Task marked as completed!")
        else:
            print("Task not found.")

    elif choice == "5":
        print("\nStudy Progress:")
        show_progress(get_tasks())

    elif choice == "6":
        print("Thank you for using Student Study Task Manager!")
        break

    else:
        print("Invalid choice. Please try again.")
```

SCREENSHOT OF OUTPUTS

SCREENSHOT 7 – Main Menu

```
===== Student Study Task Manager =====
1. Add Task
2. View Tasks
3. Search Task
4. Mark Task Completed
5. View Progress
6. Exit
```

SCREENSHOT 8 – Add Task

```
Enter your choice: 1
Enter your study task: complete python assignment
Task added successfully!
```

SCREENSHOT 9 – View Tasks

```
Enter your choice: 2

Your study tasks:
- complete python assignment [Pending]
```

SCREENSHOT 10 – Search Task

```
Enter your choice: 3
Enter a keyword to search: python
Search results:
- complete python assignment
```

SCREENSHOT 11 – Complete Task

```
Enter your choice: 4
Enter the task you completed: complete python assignment
Task marked as completed!
```

SCREENSHOT 12 – View Progress

```
Enter your choice: 5

Study Progress:
Total tasks: 1
Completed tasks: 1
Progress: 100.0 %
```

SCREENSHOT 13 – Exit

```
Enter your choice: 6
Thank you for using Student Study Task Manager!
tamannaarora@Tamannas-MacBook-Air Student Study Task Manager %
```

TESTING APPROACH

The project was tested using both manual testing of the command-line menu and automated unit tests using Python's built-in unittest framework.

Automated Tests

• *Test that a newly created task starts as pending*

.

• *Test that a valid task title is accepted.*

• *Test that an empty task title is rejected.*

Test Command

```
python3 -m unittest discover -s tests -v
```

Manual Tests

• *Add task*

• *View tasks*

• *Search task*

• *Mark task completed*

• *View progress*

• *Exit application*

CHALLENGES FACED

• *Organizing the project into separate Python modules.*

• *Handling imports between main.py and the modules package.*

• *Adding input validation for task titles.*

• *Keeping the program functional after restructuring the code.*

• *Setting up automated tests and running them from the project directory.*

• *Designing simple diagrams to document the project architecture and workflow.*

LEARNING AND KEY TAKEAWAYS

• *Learned how to create and use Python classes and objects.*

• *Learned how to divide a program into multiple modules.*

• *Learned how lists can be used to store and manage tasks.*

• *Improved understanding of functions, loops, and conditional statements.*

• *Learned the importance of validating user input.*

• *Learned basic automated testing using unittest*

• *Learned how software design diagrams document the structure and behavior of a project.*

• *Learned how README and project documentation support a software project.*

FUTURE ENHANCEMENTS

• *Add permanent storage so tasks remain available after the program closes.*

• *Add task priorities and categories.*

• *Add due dates and reminders.*

• *Add a graphical user interface.*

• *Add more detailed progress reports.*

• *Add editing and deleting of existing tasks.*

REFRENCES

• *VITyarthi Project Guidelines provided for the project requirements and report structure.*

• *Python documentation and standard Python programming concepts used during implementation.*

• *Python unittest framework used for automated testing.*